

Snake VLA vs WAM Research Benchmark

1. Project Goal

Build a small, fully local research benchmark to compare two types of models:

1. VLA baseline

2. Input: current visual observation + robot/snake state + language instruction

3. Output: next action directly

4. WAM / World-Model baseline

5. Input: current visual observation + robot/snake state + candidate action

6. Predict: future state over several steps

7. Use the predicted future to select the next action

The main research question is:

Does explicitly modeling future world dynamics improve generalization to unseen objects and unseen visual environments compared with directly predicting actions?

The benchmark should be small enough to train completely on a local RTX 5060 Ti 16GB GPU.

This is **not** intended to reproduce the full-scale DreamZero or $\pi 0$ architectures. It is a controlled toy experiment designed to isolate the difference between:

Direct action prediction

and

World modeling + action planning

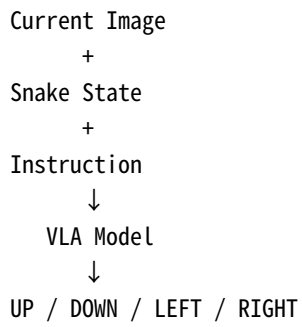
The environment is a 2D Snake game so that all dynamics are deterministic and ground truth is available.

2. Core Experimental Idea

The same Snake environment and the same base training data must be used for both models.

The only major difference should be the learning objective.

VLA



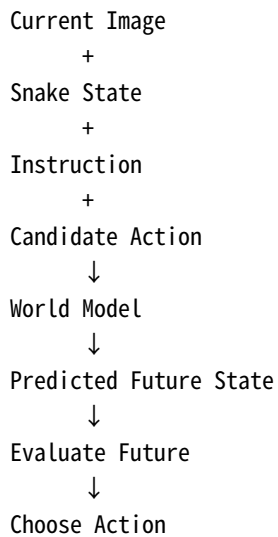
The VLA learns:

$$P(a_t \mid o_t, s_t, l)$$

where:

- o_t : current image
- s_t : current snake state
- l : language instruction
- a_t : next action

WAM



The WAM learns:

$$P(s_{t+1:t+H} \mid o_t, s_t, l, a_t)$$

At inference time, evaluate:

UP
DOWN
LEFT
RIGHT

and predict what will happen after each candidate action.

Then choose the action whose predicted future is best.

3. Research Hypothesis

The benchmark should test the following hypothesis:

A model that explicitly learns action-conditioned future dynamics should have better OOD generalization than a model that only learns direct action prediction.

The expected advantages of the WAM should be most visible in:

- unseen food appearance
- unseen visual backgrounds
- combined object + scene distribution shift
- situations requiring collision avoidance
- situations where the correct action depends on future consequences

The experiment must not assume that WAM will always outperform VLA.

The benchmark should report the actual results.

4. Environment

Implement a deterministic 2D Snake environment.

Recommended initial configuration:

Grid size: 20 × 20
Observation size: 224 × 224 RGB
Action frequency: 5–10 Hz
Actions: UP
DOWN
LEFT
RIGHT

The game should have two modes:

Manual Mode
AI Mode

5. Manual Control

The game must support human control using:

W → UP
A → LEFT
S → DOWN
D → RIGHT

Manual mode is useful for:

- debugging
- visual inspection
- collecting example trajectories
- testing environment dynamics
- demonstrating the benchmark

Do not implement AI control by simulating OS-level keyboard presses.

Instead, create a clean environment API:

```
game.step(action)
```

For example:

```
game.step("UP")  
game.step("DOWN")  
game.step("LEFT")  
game.step("RIGHT")
```

Keyboard input should simply translate WASD into these actions.

6. AI Control Interface

The environment must also support programmatic control.

Recommended interface:

```
obs = game.reset()

while not game.done:
    action = model.predict(obs)
    obs, reward, done, info = game.step(action)
```

The model process should not directly manipulate the game internals.

The interface should make it possible to later run:

```
Game Process
  ↓
Observation
  ↓
Model Process
  ↓
Action
  ↓
Game Process
```

The first implementation may keep everything in one Python process for simplicity.

However, the code architecture should make a separate model process possible later.

7. Environment Rules

The environment should implement the actual game physics.

The model should NOT be given these rules through language instructions.

The environment handles:

- wall collision
- self collision
- food collision
- snake movement
- food respawning
- game over
- reward
- score

For example:

```
collision with wall → game over
collision with body → game over
```

```
eating food      → reward +1  
normal movement → small or zero reward
```

Do not put instructions such as:

```
"Do not hit the wall."  
"Do not hit yourself."  
"Avoid obstacles."
```

into the language prompt.

The purpose of the experiment is for the model to learn action consequences from data.

8. Snake State Representation

The visual observation contains the complete rendered game.

The structured state should contain only information about the snake itself.

Recommended state:

```
head position  
current direction  
tail/body representation  
turning points
```

A compact representation can use:

```
head_x  
head_y  
direction  
turning_points
```

The number of turning points should have a fixed maximum.

For example:

```
MAX_TURNING_POINTS = 10
```

If fewer than 10 turning points exist, pad the remaining values.

Example:

```
[
  head_x,
  head_y,
  direction,
  turn_1_x,
  turn_1_y,
  turn_2_x,
  turn_2_y,
  ...
]
```

The exact representation may be changed if necessary, but it must remain:

1. deterministic
2. fixed-size
3. easy to serialize
4. identical for VLA and WAM

9. Important Information-Leakage Rule

Do NOT include the food position or food category in the structured state.

The model should obtain food information from the image.

Otherwise the model could bypass the visual understanding problem.

The state should describe the snake, not the target.

Therefore:

```
Image:
  snake
  food
  background

State:
  snake position
  snake direction
  snake body / turning points

Instruction:
  target description
```

10. Food Categories

The benchmark needs three food categories:

A
B
C

Training should contain:

A
B

while C should be used as an OOD category.

However, avoid making C simply an arbitrary unseen token.

For example, if the model sees:

"Eat A"
"Eat B"

during training and then receives:

"Eat C"

at test time, a small learned text embedding has no reason to understand what C means.

Therefore the implementation should support two levels of experiment.

Experiment A — Simple OOD

Use:

A = visual appearance 1
B = visual appearance 2
C = visual appearance 3

This measures robustness to unseen visual appearance.

Experiment B — Compositional Semantic OOD

Prefer a compositional description such as:

red circle
blue square
yellow triangle

or another set of attributes.

Train on combinations such as:

red circle
red square
blue circle

and test on:

blue square

This creates a more meaningful compositional OOD experiment.

The architecture should support natural-language instructions so that this experiment can be added later.

The initial implementation may start with the simple A/B/C version.

11. Food Appearance

Food should have clearly distinguishable visual appearances.

For example:

A:
red circle

B:
blue square

C:
yellow triangle

The exact visual design can be changed.

The important requirements are:

- A, B and C have identical game physics
- only their visual/semantic identity changes

- C does not appear in training for the OOD experiment
- the model cannot identify the target from structured state

12. Background / Scene Distribution

The environment should support multiple background styles.

Training

Use:

Plain background

OOD

Use:

Striped background

Optionally support:

checkerboard
noise
different grid patterns
different texture

The first experiment should use only:

Plain
Striped

to keep the experiment interpretable.

13. Main 2×2 Benchmark

The main benchmark should be:

Object	Scene	Distribution
A/B	Plain	ID
A/B	Striped	Scene OOD

Object	Scene	Distribution
C	Plain	Object/Semantic OOD
C	Striped	Combined OOD

This gives four evaluation conditions:

ID
Scene OOD
Object OOD
Object + Scene OOD

Both VLA and WAM must be evaluated on exactly the same episodes.

14. Dataset Generation

Do not manually record thousands of games.

The game environment should automatically generate trajectories.

Use an expert policy to control the snake.

The expert does not need to be a neural network.

A simple heuristic is sufficient:

1. identify the target food
2. move toward the food
3. avoid immediate wall collision
4. avoid immediate self collision
5. prefer safe actions
6. continue until food is eaten or the episode ends

The expert should generate diverse trajectories rather than always taking the exact same path.

Add controlled randomness where appropriate.

15. Dataset Size

Start with:

Training episodes:
5,000–10,000

Average episode length:
100–300 steps

Expected transitions:
~1M–3M

Do not generate millions of samples before confirming that the entire pipeline works.

Recommended development order:

```
10 episodes
  ↓
100 episodes
  ↓
1,000 episodes
  ↓
5,000 episodes
  ↓
10,000 episodes
```

The first milestone should be a tiny dataset that can train successfully end-to-end.

16. Dataset Format

Each transition should contain:

```
frame_t
state_t
instruction_t
action_t
frame_t+1
state_t+1
reward_t
done_t
```

For WAM, future trajectories should also be accessible:

```
state_t+1
state_t+2
...
state_t+H
```

where:

H = 5 or 10

Start with:

H = 5

to keep training simple.

17. Dataset Storage

Do not create millions of individual PNG files if avoidable.

Use a structured dataset format.

Possible options:

HDF5
NPZ
WebDataset
Parquet + image storage

Choose a format that provides:

- fast sequential reading
- random access
- efficient storage
- easy PyTorch integration

For the first implementation, a simple and reliable format such as HDF5 or an equivalent local format is preferred.

The dataset loader should expose:

```
sample = dataset[idx]
```

with:

```
sample["image"]  
sample["state"]  
sample["instruction"]  
sample["action"]
```

```
sample["next_state"]  
sample["future_states"]
```

18. Dataset Split

Do not randomly split individual frames from the same episode across train and test.

Split by episode.

Recommended:

```
Train:  
80%  
  
Validation:  
10%  
  
Test:  
10%
```

However, OOD test sets must be generated separately.

Use:

```
train/  
  A_B/  
  plain/  
  
val/  
  A_B/  
  plain/  
  
test/  
  ID/  
  scene_ood/  
  object_ood/  
  combined_ood/
```

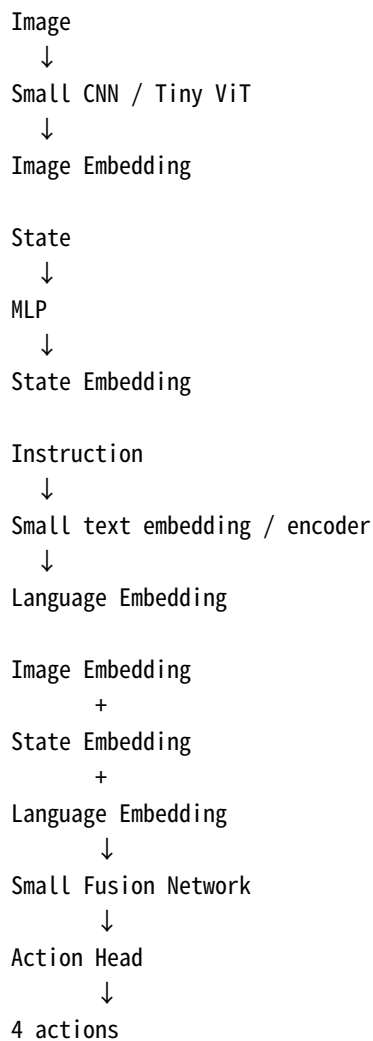
19. VLA Model

The first VLA should be intentionally small.

Do NOT use the actual π_0 architecture for the first experiment.

The goal is to establish a clean baseline.

Recommended architecture:



Target model size:

~5M–20M parameters

The model should fit comfortably into 16GB VRAM.

20. VLA Training Objective

Train using action classification.

For each training sample:

```

image_t
state_t
instruction_t
    ↓
model
    ↓
action logits

```

Use cross entropy:

$$L_{action} = -\log P(a_t|o_t, s_t, l)$$

The model predicts:

```

UP
DOWN
LEFT
RIGHT

```

21. WAM Model

The first WAM should also be small.

Do not implement a full video diffusion model.

The first goal is to test whether explicit future prediction helps.

Recommended architecture:

```

Image
  ↓
Image Encoder
  ↓
Image Embedding

State
  ↓
State Encoder
  ↓
State Embedding

Instruction
  ↓
Language Encoder
  ↓

```

Language Embedding

Candidate Action



Action Embedding

All embeddings



Fusion / Transformer



Future State Decoder



$s(t+1 \dots t+H)$

Target model size:

~10M-30M parameters

22. WAM Training Objective

The WAM should learn:

$$(o_t, s_t, l, a_t) \rightarrow s_{t+1:t+H}$$

For the first implementation, predict future structured states rather than future pixels.

For example:

```
input:
  image_t
  state_t
  instruction
  action_t

output:
  state_t+1
  state_t+2
  ...
  state_t+5
```

Use an appropriate state prediction loss.

For continuous coordinates:

MSE / Smooth L1

For direction:

Cross entropy

For collision/game-over:

Binary cross entropy

The implementation may combine these into:

$$L_{world} = L_{position} + \lambda_1 L_{direction} + \lambda_2 L_{collision}$$

23. WAM Action Selection

At inference time, do not directly predict the action.

Instead evaluate all candidate actions:

UP
DOWN
LEFT
RIGHT

For each action:

(current observation, current state, instruction, action)



WAM



future trajectory

Then calculate a planning score.

The score should favor:

- reaching the target
- avoiding collision
- remaining alive
- reducing distance to target

For example:

```
score =  
    target_progress  
    - collision_penalty  
    + survival_bonus
```

The exact scoring function should be configurable.

Then:

```
best_action = argmax(score)
```

24. Important Fairness Constraint

VLA and WAM must use:

- the same environment
- the same training episodes
- the same images
- the same instructions
- the same snake states
- the same train/validation split

Do not give WAM additional training data.

Otherwise an improvement cannot be attributed cleanly to world modeling.

The primary experimental difference should be:

```
VLA:  
learn action directly  
  
WAM:  
learn future dynamics and plan using them
```

25. Real-Time Inference Loop

The system should support:

```
Game  
↓  
capture current frame  
↓
```

```
extract current snake state
↓
send observation to model
↓
model predicts action
↓
game.step(action)
↓
render next frame
↓
repeat
```

For VLA:

```
frame + state + instruction
↓
action
```

For WAM:

```
frame + state + instruction
↓
evaluate 4 candidate actions
↓
future predictions
↓
planning
↓
action
```

The game should display:

```
FPS
step count
score
current mode
model name
target
```

26. Manual / AI Toggle

The game should support switching between:

```
Manual  
VLA  
WAM
```

For example:

```
M → Manual  
V → VLA  
W → WAM
```

Avoid conflicts between movement keys and mode-selection keys.

The exact key bindings can be adjusted.

27. Visualization

The environment should be visually simple.

Example:

```
+-----+  
|       |  
|  food  |  
|       |  
|  snake  |  
|       |  
+-----+
```

```
Target: Eat A  
Score: 3  
Mode: WAM  
FPS: 8.4
```

The visual design must be deterministic and configurable.

The benchmark should allow:

```
background="plain"  
background="striped"  
food_type="A"  
food_type="B"  
food_type="C"
```

28. Experiment Configuration

All major parameters should be configurable through YAML or command-line arguments.

For example:

```
grid_size: 20
image_size: 224
max_steps: 300

food_types:
- A
- B

background: plain

action_horizon: 5

seed: 42
```

Do not hard-code experimental parameters inside model code.

29. Reproducibility

All experiments must support deterministic seeds.

Use:

```
Python random seed
NumPy seed
PyTorch seed
environment seed
```

Every generated dataset should record:

```
seed
environment configuration
dataset version
model configuration
```

30. Recommended Project Structure

Create a clean repository such as:

```
snake-vla-wam/  
|  
|—— README.md  
|—— requirements.txt  
|—— pyproject.toml  
|  
|—— configs/  
|   |—— env.yaml  
|   |—— dataset.yaml  
|   |—— vla.yaml  
|   |—— wam.yaml  
|  
|—— snake/  
|   |—— __init__.py  
|   |—— env.py  
|   |—— renderer.py  
|   |—— state.py  
|   |—— actions.py  
|   |—— expert.py  
|   |—— game.py  
|  
|—— data/  
|   |—— generator.py  
|   |—— dataset.py  
|   |—— schema.py  
|   |—— split.py  
|  
|—— models/  
|   |—— image_encoder.py  
|   |—— state_encoder.py  
|   |—— language_encoder.py  
|   |—— vla.py  
|   |—— wam.py  
|  
|—— training/  
|   |—— train_vla.py  
|   |—— train_wam.py  
|   |—— losses.py  
|   |—— utils.py  
|  
|—— evaluation/  
|   |—— evaluate.py  
|   |—— metrics.py  
|   |—— ood.py  
|   |—— planner.py
```

```
|
|—— inference/
|   |—— run_vla.py
|   |—— run_wam.py
|   |—— interactive.py
|
|—— scripts/
|   |—— generate_dataset.py
|   |—— train_vla.sh
|   |—— train_wam.sh
|   |—— evaluate.sh
|
|—— checkpoints/
|—— datasets/
|—— logs/
|—— results/
```

The exact structure can be changed if there is a better engineering solution, but keep environment, data, models, training and evaluation separated.

31. Development Phases

Do NOT try to implement everything simultaneously.

Implement the project in the following phases.

Phase 1 — Snake Environment

Build:

```
Snake environment
Manual WASD control
Rendering
Food
Collision
Score
Reset
Step API
```

Acceptance test:

```
Human can play the game normally.
```

Phase 2 — Environment API

Implement:

```
reset()
step(action)
render()
get_state()
get_observation()
```

Acceptance test:

A Python script can control the game without keyboard input.

Example:

```
env = SnakeEnv()

obs = env.reset()

for _ in range(100):
    obs, reward, done, info = env.step("RIGHT")
```

Phase 3 — Expert Policy

Implement a simple automated Snake controller.

Acceptance test:

```
expert can successfully eat food
expert avoids obvious collisions
```

The expert does not need to be optimal.

Phase 4 — Dataset Generator

Generate a tiny dataset first:

```
10 episodes
```

Verify:

- images are correct
- state is correct
- action is correct
- next state is correct
- instructions are correct
- train/test distribution is correct

Then scale:

```
100
1000
5000
10000
```

32. Phase 5 — Dataset Inspection

Create tools to inspect random trajectories.

For each sample, visualize:

```
frame
instruction
state
action
next state
```

Also create trajectory playback.

This is important because dataset errors can invalidate the entire experiment.

33. Phase 6 — VLA Baseline

Implement the smallest working VLA.

First train on:

```
A/B
plain background
```

The model should achieve high training accuracy.

Then evaluate on:

```
ID
Scene 00D
Object 00D
Combined 00D
```

Save:

```
action accuracy
success rate
score
survival time
steps to food
```

34. Phase 7 — WAM

Implement future-state prediction.

First ignore planning.

Train:

```
state_t + action
      ↓
state_t+1
```

Verify that future prediction works.

Then increase:

```
H = 5
```

and predict multiple future states.

Only after prediction works should planning be implemented.

35. Phase 8 — WAM Planning

Implement:

```
candidate action generation
  ↓
future prediction
  ↓
trajectory scoring
  ↓
argmax action
```

Evaluate all four actions.

The planner should be deterministic given a model checkpoint and seed.

36. Phase 9 — Full Comparison

Run exactly the same test episodes for both models.

Generate a table:

Model	ID	Scene OOD	Object OOD	Combined OOD
VLA				
WAM				

Metrics should include:

```
Success Rate
Average Score
Average Survival Steps
Target Completion Rate
Action Accuracy
```

37. Future Prediction Evaluation

For WAM, separately evaluate world-model quality.

Measure:

```
1-step prediction accuracy
5-step prediction accuracy
collision prediction accuracy
food-reaching prediction accuracy
```

Do not rely only on pixel metrics.

Since Snake is a discrete environment, structured state prediction is more meaningful.

38. Main Research Metrics

The most important metric is:

Success Rate

Percentage of episodes where the target food is successfully eaten.

Also report:

OOD Generalization Gap

$$Gap = Performance_{ID} - Performance_{OOD}$$

A smaller gap means better OOD robustness.

For example:

VLA:
ID = 95%
OOD = 55%
gap = 40%

WAM:
ID = 92%
OOD = 72%
gap = 20%

This would support the hypothesis that world modeling improves generalization.

39. Required Ablation Studies

Once the main experiment works, implement several ablations.

Ablation 1 — No structured state

image + instruction

instead of:

image + state + instruction

Ablation 2 — No language

Remove the instruction.

This tests whether the model is simply following the nearest food.

Ablation 3 — WAM without planning

Use the WAM's action head directly instead of evaluating future trajectories.

This tests whether the benefit comes from:

world prediction

or merely from a different architecture.

Ablation 4 — Short vs long horizon

Compare:

H = 1
H = 5
H = 10

This tests whether longer future reasoning helps.

40. Important Experimental Control

The model should not be able to cheat.

Check for:

Food position leakage

Food position must not be in structured state.

Target leakage

If the target is explicitly represented in the state, the language component becomes unnecessary.

Background leakage

Train and test backgrounds must differ in controlled ways.

Episode leakage

Do not put frames from the same trajectory into both training and test.

Action leakage

Do not accidentally include the ground-truth action in the model input.

41. Optional Future Extension: Video World Model

Once the state-based WAM works, extend it to actual future-frame prediction.

Instead of:

```
state_t + action
      ↓
future state
```

implement:

```
frame_t
state_t
action
      ↓
future frames
```

For example:

```
frame_t
      ↓
frame_t+1
frame_t+2
frame_t+3
frame_t+4
frame_t+5
```

This becomes much closer conceptually to a real WAM.

However, this is a **second-stage experiment**.

Do not make video diffusion part of the initial implementation.

42. Optional Future Extension: Real VLA / WAM

After the toy benchmark is working, investigate adapting real systems such as:

$\pi 0$ / $\pi 0.5$
DreamZero

to the Snake environment.

This should be treated as a separate stage.

The current benchmark should remain independent from those large models.

43. Hardware Target

The primary development machine has:

GPU:
NVIDIA RTX 5060 Ti 16GB

Target:
local training

The project should be optimized for a single 16GB GPU.

Use:

PyTorch
CUDA
mixed precision / BF16 or FP16 when appropriate

Avoid architectures that require multi-GPU training.

The initial models should be small enough that training is practical on a single GPU.

44. Training Configuration

The training scripts should expose:

```
batch size
learning rate
number of epochs
gradient accumulation
image size
hidden dimension
number of layers
future horizon
checkpoint frequency
evaluation frequency
seed
```

Example:

```
python training/train_vla.py
--config configs/vla.yaml
```

and:

```
python training/train_wam.py
--config configs/wam.yaml
```

45. Logging

Every training run should save:

```
training loss
validation loss
action accuracy
future prediction loss
learning rate
epoch
step
```

Use a simple logging solution such as:

```
TensorBoard
```

or another lightweight local logger.

Do not introduce unnecessary external services.

46. Checkpoints

Save:

```
best checkpoint
latest checkpoint
configuration
training statistics
```

Example:

```
checkpoints/
  vla_best.pt
  vla_latest.pt
  wam_best.pt
  wam_latest.pt
```

The checkpoint must contain enough information to reproduce inference.

47. Evaluation Reproducibility

Evaluation must use fixed test seeds.

For example:

```
test_seed = 1000
```

The same test episodes must be used for:

```
VLA
WAM
all ablations
```

This makes comparisons fair.

48. Visualization of Results

Generate:

Figure 1

Example ID trajectory.

Figure 2

Scene OOD trajectory.

Figure 3

Object OOD trajectory.

Figure 4

Combined OOD trajectory.

Figure 5

VLA vs WAM success rate.

Figure 6

VLA vs WAM OOD performance.

Figure 7

WAM predicted future trajectory vs ground truth.

The predicted future trajectory visualization is especially important.

Example:

Current state

↓

WAM prediction:

UP → predicted safe

DOWN → collision

LEFT → safe but far from target

RIGHT → target reached

↓

choose RIGHT

49. Final Deliverables

The project is complete when it provides:

Environment

- playable Snake
- WASD control
- deterministic environment
- programmatic action API
- configurable food
- configurable background

Dataset

- automatic trajectory generation
- train/validation/test split
- ID and OOD datasets
- reproducible generation

VLA

- training
- validation
- inference
- checkpointing
- evaluation

WAM

- future-state prediction
- action-conditioned prediction
- candidate-action planning
- inference
- evaluation

Benchmark

- ID evaluation
- Scene OOD
- Object OOD
- Combined OOD
- quantitative comparison
- visualization

50. Recommended First Milestone

Do not start by implementing the neural networks.

The first milestone should be:

```
Snake game
  ↓
Expert controller
  ↓
10 generated episodes
  ↓
Dataset files
  ↓
Visualization of trajectories
```

After this works:

```
10
  ↓
100
  ↓
1,000
  ↓
5,000–10,000
```

Only then start model training.

51. Coding Philosophy

Keep the project simple and research-oriented.

Priorities:

1. Correctness
2. Reproducibility
3. Clear experimental separation
4. Easy debugging
5. Small models
6. Fast iteration

Avoid unnecessary complexity.

Do not:

- download huge pretrained VLA models for the first version
- implement diffusion models initially
- implement multi-GPU training
- build a complicated robotics simulator
- introduce unnecessary dependencies
- optimize before the benchmark works

The first goal is to answer one question clearly:

In the same Snake environment and with the same training data, does learning action-conditioned future dynamics improve OOD generalization compared with directly predicting actions?

52. Codex Implementation Instructions

Implement the project incrementally.

After each phase:

1. run the relevant tests
2. verify the output
3. fix errors
4. only then continue

Do not create placeholder functions for core functionality.

The final repository should be executable from a clean environment.

Provide:

```
README.md
installation instructions
dataset generation instructions
training instructions
evaluation instructions
interactive demo instructions
```

The README should contain the complete workflow:

```
# 1. install

# 2. test environment

# 3. generate dataset
```

4. train VLA

5. train WAM

6. evaluate

7. run interactive demo

The implementation should prioritize a **working end-to-end experiment** over architectural sophistication.

The final result should allow a researcher to run:

```
generate dataset
  ↓
train VLA
  ↓
train WAM
  ↓
evaluate both
  ↓
compare ID / OOD
  ↓
visualize results
```

without manually modifying source code.